This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

Detecting Non-Optimal Decisions of Embodied Agents via Diversity-Guided Metamorphic Testing

Wenzhao Wu, Yahui Tang, Mingfei Cheng, Wenbing Tang,
Yuan Zhou, *Member, IEEE*, and Yang Liu, *Senior Member, IEEE*

***Abstract*—As embodied agents advance toward real-world deployment, ensuring optimal decisions becomes critical for resource-constrained applications. Current evaluation methods focus primarily on functional correctness, overlooking the non-functional optimality of generated plans. This gap can lead to significant performance degradation and resource waste. We identify and formalize the problem of Non-optimal Decisions (NoDs), where agents complete tasks successfully but inefficiently. We present `NoD-DGMT`, a systematic framework for detecting NoDs in embodied agent task planning via diversity-guided metamorphic testing. Our key insight is that optimal planners should exhibit invariant behavioral properties under specific transformations. We design four novel metamorphic relations capturing fundamental optimality properties: position detour suboptimality, action optimality completeness, condition refinement monotonicity, and scene perturbation invariance. To maximize detection efficiency, we introduce a diversity-guided selection strategy that actively selects test cases exploring different violation categories, avoiding redundant evaluations while ensuring comprehensive diversity coverage. Extensive experiments on the AI2-THOR simulator with four state-of-the-art planning models show that `NoD-DGMT` achieves an average violation detection rate of 31.9%, a 16.8% relative improvement over the strongest of seven baselines, while the diversity-guided filter further improves detection by 4.3% and diversity by 3.3 cluster pairs on average, at only about 5% computational overhead. In addition, `NoD-DGMT` remains effective under sensor noise and actuator latency, and generalizes to the outdoor MetaUrban simulator, confirming its robustness and cross-domain applicability.**

***Index Terms*—Decision Optimality, Embodied Agents, Metamorphic Testing, Task Planning, Violation Diversity**

I. Introduction

While foundation models, such as Large Language Models (LLMs) and Vision-Language-Action (VLA) models, have demonstrated impressive capabilities in natural language understanding, multimodal reasoning, and content generation, they remain fundamentally disembodied and lack the ability to directly interact with the physical world [1]. This trend has led to the emergence of embodied AI [2]–[4], where embodied agents are equipped with physical forms such as robots, enabling them to actively perceive, learn from, and interact with the physical environment. Consequently, embodied agents are widely regarded as a pivotal step toward achieving artificial general intelligence, given their potential in various real-world applications [5]–[9], such as household services, industrial manufacturing, healthcare, and autonomous driving.

In embodied agents, task planning constitutes a fundamental capability that encompasses the decision-making processes required to decompose complex task objectives into executable action sequences [10], [11]. This involves sophisticated multi-dimensional reasoning, wherein agents must interpret natural language instructions, accurately assess environmental states and constraints, and effectively coordinate interdependencies among sequential subtasks [12]. The resulting action plans must be both computationally feasible and contextually appropriate for successful task execution. Recently, numerous studies have explored leveraging LLMs for embodied task planning, demonstrating promising performance and enhanced adaptability [10], [13]–[15]. As these agents become increasingly deployed in safety-critical and resource-sensitive scenarios, ensuring the quality and reliability of their decisions becomes paramount.

However, existing evaluation methods for embodied agents primarily focus on basic functional requirements, such as task success rates [16]–[18], trajectory length [19], [20], and execution time [10]. Although these functional metrics provide valuable insights into task completion capabilities, they fail to capture crucial non-functional requirements essential for real-world deployment. *One important yet underexplored aspect is optimality, which is the ability to make decisions that minimize resource consumption and maximize efficiency through the most effective means possible.* While Non-optimal Decisions (NoDs) do not compromise task completion, they can result in substantial cost increases and resource inefficiencies in practical scenarios. For instance, suboptimal planning in household service robots may lead to excessive energy consumption and user frustration due to prolonged operation times. Therefore, developing a comprehensive framework to systematically detect and evaluate NoDs is imperative for ensuring practical utility and cost-effectiveness in resource-constrained deployment scenarios.

Systematically detecting and evaluating NoDs in embodied agents presents several significant challenges. First, unlike functional testing with explicit criteria (e.g., task completion rates), optimality assessment lacks clear oracles for determin-

W. Wu and Y. Liu are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (E-mail: wenzhao.wu@ntu.edu.sg, yangliu@ntu.edu.sg).

Y. Tang is with the School of Computer, Chongqing University of Posts and Telecommunications, Chongqing, 400065, P. R. China (E-mail: tangyh@cqupt.edu.cn).

M. Cheng is with the School of Computing and Information Systems, Singapore Management University, Singapore 188065 (E-mail: mfcheng.2022@smu.edu.sg).

W. Tang is with the College of Information Engineering, Northwest A&F University, Yangling, Shaanxi 712100, P. R. China (E-mail: wenbingtang@nwafu.edu.cn).

Y. Zhou is with the School of Computer Science and Technology, Zhejiang Sci-Tech University, Hangzhou, Zhejiang 310018, P. R. China (E-mail: yuanzhou@zstu.edu.cn).

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

ing the "optimal" action sequence across diverse scenarios and contexts. Second, ensuring comprehensive NoD detection is challenging, as evaluation processes may repeatedly identify similar suboptimal patterns while overlooking other violation categories, thereby limiting coverage comprehensiveness. Third, while functional testing relies on direct input-output comparisons, optimality evaluation requires establishing meaningful comparison criteria and appropriate metrics to assess the relative performance of different decision sequences.

To address these challenges, we propose NoD-DGMT, a novel Diversity-Guided Metamorphic Testing framework designed to detect NoDs in embodied agent task planning. Specifically, we first identify and formalize four Metamorphic Relations (MRs) that capture the invariant properties of optimal decisions, enabling optimality assessment without explicit oracles. Second, we define diversity metrics and propose a diversity-guided selection strategy that filters and prioritizes test cases with greater variations to ensure comprehensive evaluation coverage across different types of potential NoD patterns. Finally, we develop a Metamorphic Testing (MT)-based detection framework that leverages the identified MRs and diverse test cases to automatically evaluate decision optimality and quantify violation severity.

We conduct comprehensive experiments to evaluate the effectiveness and efficiency of NoD-DGMT using the AI2-THOR simulator [21]. The results show that NoD-DGMT effectively detects NoDs across varying task complexities and four different planners, with violation rates ranging from 18.7% to 67.2%. Compared with seven baseline methods, NoD-DGMT achieves the highest average detection rate of 31.9%, a 16.8% relative improvement over the strongest of seven baselines. Ablation studies further confirm the contribution of the diversity-guided filter, showing that it improves the detection efficiency by 4.3% while enhancing diversity coverage, with only about 5% computational overhead. Beyond the standard evaluation setting, NoD-DGMT retains a meaningful fraction of its detection capability under injected sensor noise and actuator latency, and generalizes to the outdoor MetaUrban simulator, demonstrating its robustness and generalizability.

The main contributions of this paper are as follows.

- We propose NoD-DGMT, the first MT-based framework to evaluate the optimality of embodied agent decisions.
- We introduce four innovative MRs and a diversity-guided test case filter to systematically and efficiently discover diverse NoDs.
- We conduct extensive evaluations across multiple planners, task complexities, simulators, and baseline methods to demonstrate the effectiveness, efficiency, robustness, and generalizability of NoD-DGMT.
- To ensure reproducibility, all code and test cases are publicly available at https://nod-dgmt.github.io/project/.

The rest of this paper is organized as follows. Section II introduces the background. Section III presents the motivation and problem statement. The detailed procedures of NoD-DGMT are provided in Section IV. Experiments are conducted in Section V to demonstrate the effectiveness, efficiency, robustness, and generalizability of our approach. Section VI discusses the threats to the validity of our study. Section VII summarizes the related work. Finally, Section VIII presents the discussion and conclusion.

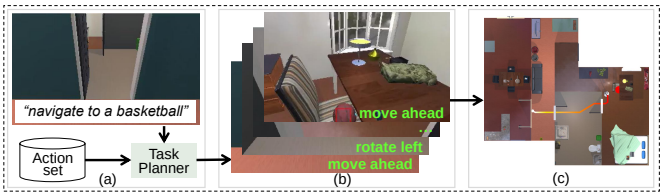


Fig. 1. The task planning process of embodied agents.

II. Background

A. Embodied Task Planning

Embodied task planning is a process to generate a sequence of executable actions to accomplish the given goal within a physical environment [10], [18]. As shown in Fig. 1(a), the planning process begins when the task planner receives a natural language instruction $\mathcal{L}$ from the user, such as *"navigate to a basketball."* Then, the planner selects a sequence of actions from the predefined action set $\mathbf{A}$ based on its perception of the 3D interactive physical environment E. The action set $\mathbf{A}$ represents the embodied agent's capabilities, where each action $a \in \mathbf{A}$ corresponds to an atomic operation the agent can execute (e.g., *"MOVE AHEAD"*, *"MOVE BACK"*, *"ROTATE LEFT"*, *"ROTATE RIGHT"*, or *"MOVE ARM UP"*), with each action being coupled to a low-level motion controller. Formally, the task planner is a policy $\Pi : E \times \mathcal{L} \rightarrow \mathbf{A}$ that maps environment states and instructions to actions. As shown in Fig. 1(b), at each time step t, the planner selects action a_t by maximizing the likelihood of completing the given instruction $\mathcal{L}$:

$$a_t = \arg\max_{a \in \mathbf{A}} p(a|\mathcal{L}; a_0, \cdots, a_{t-1}; E). \tag{1}$$

where $\langle a_0, \cdots, a_{t-1} \rangle$ represent the actions previously selected for time steps $\langle 0, \cdots, t-1 \rangle$, respectively. A task is considered successfully completed when the given instruction $\mathcal{L}$ is satisfied at the terminal time step τ:

$$r = \begin{cases} 1, & \text{if } s_\tau \models \mathcal{L} \text{ (instruction satisfied)}, \\ 0, & \text{otherwise.} \end{cases}$$

where s_τ denotes the state of the embodied agent and the environment at time step τ. Fig. 1(c) shows the traveled trajectory of the agent when it successfully arrives at the terminal state.

B. Non-Optimal Decisions (NoDs)

In practice, for a given instruction $\mathcal{L}$ in an environment E, there often exist multiple valid action sequences that can successfully complete the task. This paper focuses on detecting NoDs to identify and evaluate the suboptimality of planning decisions. NoDs represent a category of decision-making issues where the generated action sequences successfully complete assigned tasks but exhibit suboptimal characteristics compared to more efficient alternatives. Unlike functional bugs that prevent task completion, NoDs manifest as inefficient

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

yet functionally correct solutions that achieve the desired objectives while incurring unnecessary costs.

Definition 1 (NoDs)**.** *Let $\mathbf{\Theta}(\mathcal{L}, E)$ denote the set of all valid action sequences that successfully complete task $\mathcal{L}$ in environment E. Each sequence $\theta = \langle a_0, a_1, \ldots, a_\tau \rangle$ satisfies the functional correctness criterion where $s_\tau \models \mathcal{L}$. Let $C(\theta)$ be a cost function that quantifies the quality of sequence θ, incorporating factors such as execution time, action count, energy consumption, or path length. An action sequence θ generated by an embodied agent is a NoD iff:*

$$\theta \in \mathbf{\Theta}(\mathcal{L}, E) \text{ s.t. } \exists \theta' \in \mathbf{\Theta}(\mathcal{L}, E) : C(\theta') < C(\theta). \quad (2)$$

This definition captures the fundamental characteristic that NoDs are not functional failures but rather suboptimal solutions where more efficient alternatives exist within the same problem space. In resource-constrained deployment scenarios, even minor inefficiencies can accumulate into significant operational costs over extended periods. Unlike traditional software defects that cause system crashes or incorrect outputs [22]–[24], NoDs represent a class of performance issues that compromise system efficiency while maintaining functional correctness.

C. Metamorphic Testing

Unfortunately, it is hard to evaluate whether the decision θ is an optimal one due to the lack of testing oracles, i.e., we do not know the optimal plan $\theta^* = \arg\min_{\theta'} C(\theta')$. Inspired by the success of MT in automatically detecting bugs, uncovering defects, and revealing biases in software and AI systems [25]–[29], we adopt MT to evaluate decision optimality in embodied agents without requiring explicit optimal solutions. At the core of MT lies a set of MRs, which specify necessary properties of the system under test by defining relationships between multiple inputs and their expected outputs. For example, consider a monotonically increasing function $f(x)$, a simple MR is that for any two inputs where $x_1 < x_2$, the outputs should satisfy $f(x_1) < f(x_2)$. The goal of MT is to construct source test cases and their corresponding follow-up test cases based on the MRs. These pairs of test cases are then executed using the target software, and the outputs of the source and follow-up test cases are compared to check whether their corresponding MRs are satisfied. If an MR is violated, we can conclude that a failure is detected. For example, if $f(3) > f(5)$ for the test cases 3 and 5, this discrepancy suggests an error in the function implementation, thereby indicating a failure in maintaining the monotonic property.

In this work, we apply MT to systematically detect NoDs in embodied agent task planning by formulating a set of MRs that encode the expected behavioral properties of optimal decisions. This approach enables us to determine whether a decision exhibits suboptimal behavior by checking if these MRs are violated, without requiring knowledge of the optimal decision itself. For example, one MR could specify that removing obstacles from an agent's path should not increase the number of steps in the resulting plan. If a planner violates this property, it indicates suboptimal behavior.

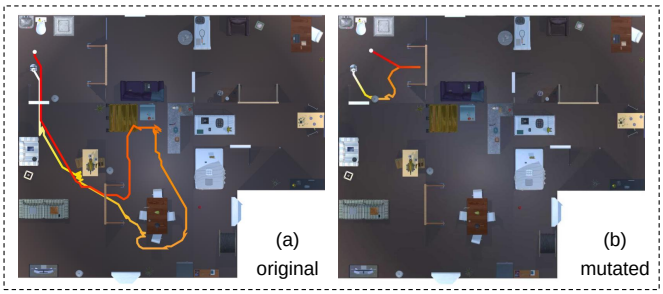


Fig. 2. A motivating example revealing an optimality issue in embodied agent decision-making. The navigation target is marked with a red circle, and the agent's terminal position is indicated by a white circle.

III. Motivation and Problem Statement

A. Motivation

This section presents a real NoD case identified in SPOC [16] and demonstrates how our method can detect it. Fig. 2 shows the trajectories traveled by the agent under the same target and environment. In the original scenario shown in Fig. 2(a), an agent is instructed to navigate to a toilet within a household environment. The agent generates a significantly circuitous path requiring 367 steps, traveling through multiple rooms with numerous redundant turns and inefficient routing decisions. To evaluate optimality, we apply a metamorphic transformation by adding an intermediate goal. Intuitively, this additional constraint should increase the path complexity and step count. However, as shown in Fig. 2(b), the agent accomplishes the modified task in only 113 steps, which is significantly fewer than the original 367 steps. The difference between the two outcomes illustrated in Fig. 2 reveals a clear violation: *the plan generated in Fig. 2(a) is non-optimal since adding an intermediate goal should not result in fewer steps than the original task*. In the following sections, we introduce how to systematically generate test cases using MT to automatically discover such NoD issues.

This motivating example clearly illustrates the advantage of our approach. By defining MRs that capture the expected properties of optimal decision-making, we can automatically generate test cases to effectively evaluate an agent's planning optimality, without the need to manually define an oracle for each individual task.

B. Problem Statement

Based on the above analysis, we formally define the research problem addressed in this paper as follows:

Problem: *Given an embodied agent with task planner Π operating in environment E, design a set of metamorphic relations $\mathcal{MR} = \{MR_1, MR_2, \ldots, MR_k\}$ that encode the expected behavioral properties of optimal decisions, and develop an MT-based testing framework to automatically detect NoDs in the generated action sequences through systematic test case generation guided by these MRs.*

Specifically, each metamorphic relation MR_i defines the expected relationship between a source test case $(\mathcal{L}, E)$ and its corresponding follow-up test case $(\mathcal{L}', E')$. The core challenge lies in formulating MRs that capture the invariant properties

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

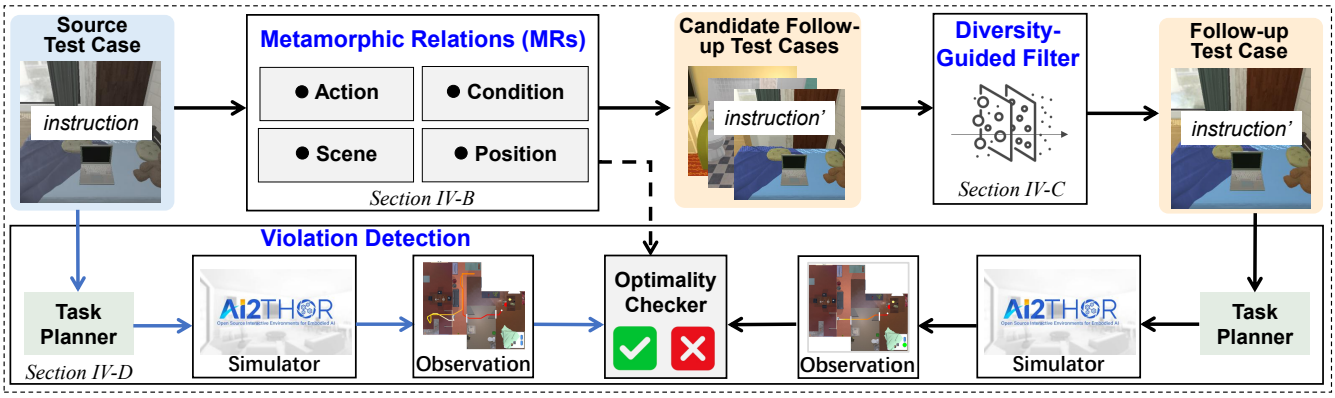


Fig. 3. The three-phase pipeline of NoD-DGMT: test case generation, diversity-guided selection, and violation detection.

of optimal planning behavior and systematically generating diverse source and follow-up test cases.

IV. Methodology

A. Overview

In this section, we introduce our method NoD-DGMT, which operates in a three-phase pipeline as illustrated in Fig. 3. First, we design and formalize four types of MRs that encode the expected behavioral properties of optimal decisions. These MRs are specifically designed for action, scene, condition, and position mutations, each capturing different aspects of optimality constraints that should hold across related planning scenarios (Section IV-B). Given a source test case, we systematically apply these MRs to generate a set of candidate follow-up test cases. Second, to ensure comprehensive evaluation coverage while maintaining efficiency, we propose a diversity-guided filter that selects the most valuable test cases from the candidate pool (Section IV-C). This selection mechanism prioritizes test cases that maximize the exploration of different types of potential optimality violations, thereby enhancing the framework's ability to detect diverse categories of NoDs while avoiding redundant evaluations. Finally, as shown in the violation detection component, NoD-DGMT executes both source and the selected follow-up test cases by feeding them into the task planner and simulator to generate corresponding action sequences and execution trajectories (Section IV-D). The optimality checker then performs comparisons of the execution results against the predefined MRs to identify violations, thereby detecting NoDs in the embodied agent's planning behavior.

B. Metamorphic Relations (MRs)

One key technical contribution of NoD-DGMT is the MRs designed for NoD detection. To this end, this section designs four MRs to evaluate decision optimality in embodied agent task planning, as illustrated in Fig. 4. The core insight behind our MR design is that optimal planners should exhibit consistent behaviors when faced with environmental or task mutations. As shown in Fig. 4, we identify four fundamental transformation categories: position, action, condition, and

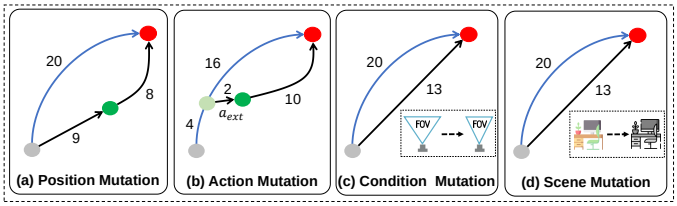


Fig. 4. Four types of mutations and their corresponding metamorphic relations for NoD detection. Gray, green, and red circles denote starting positions, intermediate waypoints, and target positions, respectively.

scene mutations, each capturing different aspects of optimality constraints that should hold across related planning scenarios.

Position Detour Suboptimality ($MR_{position}$). Given an action sequence θ that successfully completes task $\mathcal{L}$ from position A to C, creating a composite path A→B→C where the agent first moves toward intermediate position B for η steps and then continues to the original goal C should not yield a more efficient solution, i.e., $C(\theta_{A\to B}) + C(\theta_{B\to C}) \geq C(\theta_{A\to C})$. For example, if an agent generates a plan requiring 20 steps to reach a target from position A, creating a detour path that first moves toward a different object for η steps and then continues to the original goal should not result in a complete solution with fewer than 20 total steps. A violation of this MR indicates that the original sequence θ was suboptimal, suggesting the planner failed to identify that an indirect initial path could lead to a more efficient overall solution to reach the target destination.

Action Optimality Completeness (MR_{action}). Given an action sequence $\theta = \langle a_0, a_1, \ldots, a_\tau \rangle$ that successfully completes task $\mathcal{L}$, replacing the planned action a_t at time step $t \in [1, \tau - 1]$ with an extraneous action $a_{ext} \in \mathbf{A} \setminus \{a_t\}$ and re-planning the subsequent actions should not yield a more efficient solution, i.e., $C(\theta') \geq C(\theta)$ where $\theta' = \langle a_0, \ldots, a_{t-1}, a_{ext} \rangle \circ \Pi(s'_t, \mathcal{L})$. For example, if an agent generates a navigation plan θ requiring 20 steps, replacing the action at step $t = 4$ with an unrelated action like *"MOVE ARM UP"* and continuing with optimal planning from the new state should not yield a complete solution with fewer than 20 total steps. A violation of this MR indicates that the original sequence θ lacks optimality completeness, suggesting the presence of NoDs where the planner failed to identify more

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

efficient alternatives.

Condition Refinement Monotonicity ($MR_{condition}$). Given an action sequence θ that successfully completes task $\mathcal{L}$ with completion condition Ω^o, refining the completion condition to a more restrictive requirement Ω^r should not yield a more efficient solution, i.e., $C(\theta') \geq C(\theta)$ where θ' completes the refined task $\mathcal{L}'$ with condition $\Omega^r \subset \Omega^o$. For example, if an agent generates a plan requiring 20 steps to *"find the pillow"* with a Field of View (FOV) of 90 degrees, changing the condition to *"pick up the pillow"* with a more restrictive FOV of 45 degrees should not result in a solution with fewer than 20 steps. A violation of this MR indicates that the original sequence θ was suboptimal, as the planner failed to recognize that achieving the more restrictive condition could lead to a more efficient overall solution.

Scene Invariance Property (MR_{scene}). Given an action sequence θ that successfully completes task $\mathcal{L}$ in environment E, modifying task-irrelevant scene elements such as object materials, lighting conditions, or decorative items should not yield a more efficient solution, i.e., $C(\theta') \geq C(\theta)$ where θ' completes the same task $\mathcal{L}$ in the modified environment E' with cosmetic changes. For example, if an agent generates a navigation plan requiring 20 steps in a room with wooden furniture and warm lighting, changing the furniture material to metal and the lighting to cool colors should not result in a solution with fewer than 20 steps. A violation of this MR indicates that the original sequence θ was suboptimal, suggesting the planner was inappropriately influenced by task-irrelevant visual features and failed to identify the truly optimal path that should remain consistent across cosmetic scene variations.

The four MRs are designed to expose complementary forms of non-optimality in embodied task planning, targeting distinct dimensions of planner behavior: spatial routing ($MR_{position}$), action selection (MR_{action}), task-condition sensitivity ($MR_{condition}$), and perceptual robustness (MR_{scene}). Rather than relying on assumptions tied to a specific simulator or application, these MRs encode general optimality expectations that hold across embodied tasks, enabling broad applicability. Note that the framework is also extensible: domain-specific criteria, such as energy consumption or collision avoidance, can be incorporated as additional MRs by defining a corresponding cost or constraint relation, without modifying the detection pipeline.

C. Diversity-Guided Filter

Given any source test case, our four MRs can generate multiple follow-up test cases through different mutation strategies. However, applying all possible mutations can result in a large number of candidate test cases, leading to computational inefficiency and potentially redundant evaluations. The basic idea is that we want to select the most informative mutations that are likely to expose different types of NoDs while avoiding redundant test cases that explore similar aspects of optimality. To address this challenge, we introduce a diversity-guided filter component that systematically selects the most valuable test cases from the candidate pool by quantifying diversity for four types of mutations.

Specifically, for $MR_{position}$, we design a filter $F_{position}$ that measures diversity based on the perpendicular distance between the intermediate waypoint and the original trajectory. Given multiple candidate intermediate positions $\mathbf{B}^c = \{B_1, B_2, \ldots, B_n\}$, the filter selects the waypoint that maximizes the *perpendicular distance* to the original path:

$$B^* = \arg\max_{B_i \in \mathbf{B}^c} Distance(B_i, \text{Path}(A \to C)). \quad (3)$$

where $Distance(B_i, \text{Path}(A \to C))$ represents the shortest perpendicular distance from waypoint B_i to the original direct path from A to C.

For the action mutation, we design a filter F_{action} that leverages LLMs to select semantically diverse actions. Given multiple candidate alternative actions $\mathbf{A}_t^c = \{a_t^1, a_t^2, \ldots, a_t^n\}$ and the current planned action a_t, we prompt the LLM to identify the action that exhibits the greatest *semantic difference*. For example, given the planned action *"MOVE AHEAD"* and candidate actions {*"MOVE BACK"*, *"ROTATE RIGHT"*, *"MOVE ARM UP"*}, if the LLM identifies *"MOVE BACK"* as the most semantically different action, we select this action as a_{ext} for the follow-up test case generation. We adopt an LLM-based selector because the action candidates are discrete and semantically rich (e.g., navigation vs manipulation), and lexical or embedding-based distances often fail to capture such functional relationships, while an LLM's commonsense reasoning [30] better reflects which substitution is genuinely extraneous (empirical evidence is provided in Section V-H).

For the condition mutation, the filter $F_{condition}$ is defined as selecting the condition mutation with the maximum *refinement divergence* from the original condition. For continuous variables, we calculate the numerical difference between conditions. For discrete variables, we leverage LLMs to identify the condition that exhibits the greatest semantic difference. For example, when modifying the FOV from 90 degrees (original condition Ω^o), we select the candidate FOV value that maximizes $|\Omega^o - \Omega_c^r|$, where Ω_c^r represents the refined condition candidates.

Finally, for MR_{scene}, its filter F_{scene} is implemented by calculating the *scene similarity*. Among all candidates, we select the case with the lowest similarity to the original scene, ensuring that the selected follow-up test case introduces the maximum visual variation while maintaining task-irrelevance. The scene similarity is computed based on visual features extracted from the environment, including object appearances, material properties, lighting conditions, and color schemes. This selection strategy maximizes the diversity of visual modifications while ensuring that the changes remain semantically irrelevant to the task objective, thereby providing the maximum potential for detecting NoDs.

D. Metamorphic Testing-based Violation Detection Framework

This section introduces the MT-based testing framework for NoD detection. As illustrated in the bottom part of Fig. 3, the violation detection component operates by executing both source and follow-up test cases through the task planner and simulator, then comparing their execution results against the

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

Algorithm 1 Metamorphic Testing-based Violation Detection

Input: Source case $(\mathcal{L}, E)$, follow-up case $(\mathcal{L}', E')$, planner Π, and a specific metamorphic relation $MR_k \in \mathcal{MR}$.
Output: Violation detection result.

1: Initialize $violated \leftarrow False$;
2: Execute source case: $\theta_s \leftarrow \Pi(\mathcal{L}, E)$;
3: Execute follow-up case: $\theta_f \leftarrow \Pi(\mathcal{L}', E')$;
4: **if** either task fails **then**
5: **return** `Task_Execution_Failed`;
6: **end if**
7: Calculate costs: $C_s \leftarrow C(\theta_s)$, $C_f \leftarrow C(\theta_f)$;
8: **if** $C_f < C_s$ **then**
9: $violated \leftarrow True$;
10: **end if**
11: **return** $\langle violated, MR_k, C_s, C_f \rangle$;

predefined MRs. Formally, given a source test case $(\mathcal{L}, E)$ and the follow-up case $(\mathcal{L}', E') = \phi(\mathcal{L}, E)$ generated by an MR transformation ϕ, rolling out the planner Π yields the source plan θ_s and the follow-up plan θ_f, whose quality is measured by a cost function $C(\cdot)$ instantiated with step count. The MR is encoded as a boolean predicate that holds if and only if the transformation does not yield a more efficient solution:

$$MR\big((\mathcal{L}, E), (\mathcal{L}', E'), \theta_s, \theta_f\big) = \begin{cases} \textit{True} & \text{if } C(\theta_f) \geq C(\theta_s), \\ \textit{False} & \text{otherwise.} \end{cases}$$

The predicate is *False* precisely when $C(\theta_f) < C(\theta_s)$, i.e., the follow-up plan is strictly cheaper than the source, revealing that θ_s was non-optimal and exposing a NoD.

Algorithm 1 presents the complete violation detection process of `NoD-DGMT`. It takes as input a source test case and its corresponding follow-up test case selected by our diversity-guided filter, the task planner under test, and the specific MR to be evaluated. The algorithm outputs a violation detection result indicating whether a NoD has been identified. The algorithm begins with initialization (Line 1), setting the violation flag to $False$ to track whether a NoD violation is detected during the execution process. Lines 2–3 describe the task execution phase. `NoD-DGMT` begins by executing both the source and follow-up test cases using the task planner Π to generate their respective action sequences θ_s and θ_f in the simulator. If either task execution fails, the algorithm immediately returns a task execution failure status (Lines 4–6), as meaningful optimality comparison cannot be performed without successful task completion. Following successful execution, the algorithm calculates the cost metrics for both action sequences using the cost function $C(\cdot)$ (Line 7). Lines 8–10 implement the optimality checker, which examines whether the transformed scenario yields superior efficiency ($C_f < C_s$). When a violation is detected, the violated flag is set to $True$ (Line 9), indicating the presence of a NoD. Finally, the algorithm returns a comprehensive result tuple containing the violation status, the specific MR tested, and both cost values.

V. Evaluation

In this section, we evaluate the effectiveness, efficiency, robustness, and generalizability of `NoD-DGMT` in NoD detection. We aim to answer the following Research Questions (RQs):

- **RQ1:** How effective is the proposed `NoD-DGMT` in detecting NoDs?
- **RQ2:** How efficient is `NoD-DGMT` in terms of computational overhead and scalability?
- **RQ3:** How does `NoD-DGMT` perform in comparison with other existing testing methods in terms of NoD detection capability?
- **RQ4:** How robust is `NoD-DGMT` to noise commonly encountered in embodied perception and execution?
- **RQ5:** How well does `NoD-DGMT` generalize across different simulators and scenarios?
- **RQ6:** How useful is the proposed diversity-guided filter in enhancing the detection capability of `NoD-DGMT`?
- **RQ7:** How do the key configuration choices of each MR affect `NoD-DGMT` performance?

To answer RQ1, we evaluate `NoD-DGMT` in detecting NoDs across various task categories with four well-trained task planning models in the AI2-THOR simulator (Section V-B). For RQ2, we decompose the per-test-case time of `NoD-DGMT` and analyze how it scales with task complexity (Section V-C). For RQ3, we compare `NoD-DGMT` with seven baseline methods (Section V-D). To answer RQ4, we inject sensor noise and actuator latency into the simulator and evaluate NoD detection under these conditions (Section V-E). For RQ5, we deploy `NoD-DGMT` in the MetaUrban simulator to evaluate its generalization to outdoor scenarios beyond the indoor environments of AI2-THOR (Section V-F). For RQ6, we perform ablation studies to compare NoD detection performance and diversity coverage across the four MRs with and without the diversity-guided filter (Section V-G). For RQ7, we justify the key configuration choices of each MR through ablation (Section V-H).

A. Experimental Settings

In the following, we describe the experimental setup, including model selection, implementation details, and evaluation metrics.

Model Selection. Our experiments employ four state-of-the-art pre-trained models from [16] as the task planner: DINOv2, SigLIP, SigLIP-Nav, and SigLIP-Det. Specifically, DINOv2, SigLIP, and SigLIP-Det are trained on the CHORES-$\mathbb{S}$ dataset [16], while SigLIP-Nav is trained on the CHORESNAV-$\mathbb{S}$ dataset [16]. SigLIP, SigLIP-Nav, and SigLIP-Det utilize SIGLIP [31] image and text encoders, while DINOv2 employs DINOV2 [32] image and text encoders. SigLIP-Det incorporates ground truth detection capabilities provided by the simulator. These models represent different approaches to visual-language understanding and navigation planning, enabling comprehensive evaluation of our NoD detection framework across diverse planning paradigms.

Implementation details. We implement four MRs in the AI2-THOR simulator. For $MR_{position}$, we assign a different target object for the planner in the first several steps (1/4 of the entire step in the source test case) and recovering

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

TABLE I
VIOLATION DETECTION RATES OF NoD-DGMT ACROSS DIFFERENT TASK COMPLEXITY AND PLANNING MODELS.

Type	Model	Position			Action			Condition			Scene		
		Long	Medium	Short	Long	Medium	Short	Long	Medium	Short	Long	Medium	Short
Slight	DINOv2	0.0%	0.0%	5.3%	5.9%	0.0%	10.4%	6.3%	8.3%	4.7%	22.2%	11.4%	6.8%
	SigLIP	7.1%	2.6%	6.8%	**19.2%**	5.3%	9.8%	3.8%	2.7%	14.3%	**29.2%**	5.6%	12.2%
	SigLIP-Nav	0.0%	5.4%	2.3%	13.6%	5.4%	10.9%	8.0%	5.3%	4.4%	5.0%	26.5%	10.9%
	SigLIP-Det	15.6%	10.2%	9.8%	14.3%	**15.3%**	**12.9%**	4.2%	10.3%	14.5%	12.5%	13.8%	9.7%
Moderate	DINOv2	0.0%	9.4%	7.9%	0.0%	8.1%	8.3%	18.8%	2.8%	2.3%	5.6%	2.9%	2.3%
	SigLIP	0.0%	10.3%	4.5%	7.7%	5.3%	2.4%	0.0%	8.1%	11.9%	4.2%	2.8%	14.6%
	SigLIP-Nav	4.5%	2.7%	2.3%	9.1%	10.8%	2.2%	4.0%	10.5%	4.4%	5.0%	2.9%	6.5%
	SigLIP-Det	0.0%	13.6%	11.5%	2.4%	1.7%	6.5%	4.2%	8.6%	16.1%	8.3%	6.9%	4.8%
Severe	DINOv2	8.7%	15.6%	18.4%	5.9%	5.4%	0.0%	18.8%	16.7%	25.6%	5.6%	22.9%	**22.7%**
	SigLIP	7.1%	12.8%	25.0%	7.7%	5.3%	2.4%	19.2%	**32.4%**	14.3%	8.3%	**27.8%**	17.1%
	SigLIP-Nav	13.6%	**29.7%**	32.6%	4.5%	2.7%	6.5%	**28.0%**	28.9%	**28.9%**	10.0%	14.7%	21.7%
	SigLIP-Det	**17.8%**	22.0%	**45.9%**	7.1%	8.5%	4.8%	14.6%	15.5%	19.4%	18.8%	20.7%	14.5%
Total	DINOv2	8.7%	25.0%	31.6%	11.8%	13.5%	18.7%	43.8%	27.8%	32.6%	33.3%	37.1%	31.8%
	SigLIP	14.3%	25.6%	36.4%	34.6%	15.8%	14.6%	23.1%	43.2%	40.5%	41.7%	36.1%	43.9%
	SigLIP-Nav	18.2%	37.8%	37.2%	27.3%	18.9%	19.6%	40.0%	44.7%	37.8%	20.0%	44.1%	39.1%
	SigLIP-Det	33.3%	45.8%	67.2%	23.8%	25.4%	24.2%	22.9%	34.5%	50.0%	39.6%	41.4%	29.0%

the original goal in the following steps. For MR_{action}, we utilize GPT-4 [30] to determine the semantic distance and substitute the action at the middle point of the original action sequence, i.e., $t = \tau/2$. We query GPT-4 with the decoding temperature set to 0, so that the semantic action selection is deterministic and reproducible. For $MR_{condition}$, we adjust the FOV of the agent's camera with scaling factors ranging from 0.75 to 1.0. For MR_{scene}, we generate different scenes through two approaches: (1) randomizing scene materials with sensible alternatives from similar material types, and (2) randomizing lighting parameters including color, hue ($0 \sim 1$), and brightness ($0.5 \sim 1.5$). We follow the official evaluation split of the CHORES-$\mathbb{S}$ OBJNAV benchmark [16] to generate the source test case set. We categorize the test set into three task complexity levels: *Short*, *Medium*, and *Long*, based on the step count of feasible reference paths provided in the dataset. To define task complexity, we rank test cases by the length of their feasible reference paths and partition the ranked cases into three equal-sized groups. We label the lower, middle, and upper groups as *Short*, *Medium*, and *Long*, respectively.

Evaluation Metrics. We measure the effectiveness of each MR through the *Violation Detection Rate (VDR)*, defined as the percentage of executed test cases in which NoD-DGMT detects a violation. We further quantify the degree of suboptimality of each violation with the *Violation Severity (VS)*, defined as the relative cost reduction:

$$VS = \frac{C(\theta_{orig}) - C(\theta_{mut})}{C(\theta_{orig})}, \tag{4}$$

where $C(\cdot)$ is a configurable cost function, and θ_{orig} and θ_{mut} denote the source and mutation plans. We instantiate $C(\cdot)$ with step count. Since an embodied agent executes discrete primitive actions of roughly comparable duration and energy, step count is approximately proportional to execution time and provides a proxy for actuation energy. VS therefore measures the fraction of time, and approximately energy, wasted by a non-optimal plan, rather than an abstract step difference. The same definition extends to resource-specific costs such as energy units or wall-clock time by instantiating $C(\cdot)$ accordingly.

To enable a fine-grained analysis, we use VS to classify violations into three levels: *Slight* ($VS \leq 10\%$), *Moderate* ($10\% < VS \leq 20\%$), and *Severe* ($VS > 20\%$); we justify these two thresholds in Section V-H.

B. RQ1: Effectiveness of NoD-DGMT

1) Detection Performance Evaluation: To evaluate the effectiveness of NoD-DGMT, we applied each MR to 400 test cases (200 source test cases and 200 follow-up test cases) across four planners. Table I shows the violation detection rates of our four MRs across different VS levels (Slight, Moderate, Severe, and Total), MR types (Position, Action, Condition, Scene), and test case lengths (Long, Medium, Short).

From Table I, it can be observed that across all cases, despite variations in detection rates among different mutation types, they all maintain a "rising with severity" trend: the detection rate of *Severe* violations is consistently the highest, followed by *Moderate* violations, and *Slight* violations have the lowest rates. For example, in Position mutation (Short-term case), SigLIP-Det achieves a 45.9% detection rate for *Severe* violations, far exceeding its 9.8% rate for *Slight* violations. This indicates that our proposed four MRs can effectively identify NoDs of varying severity, with a particular ability to detect severe optimality violations.

Among the four MRs, $MR_{condition}$ and $MR_{position}$ demonstrate prominent performance. For example, $MR_{condition}$ of SigLIP-Nav achieves detection rates of 28.0% (*Severe*, Long-term case), 10.5% (*Moderate*, Medium-term case), and 8% (*Slight*, Long-term case), while $MR_{position}$ excels in detecting *Severe* violations (e.g., 45.9% for SigLIP-Det in Short-term cases). In contrast, MR_{action} exhibits relatively lower but still meaningful detection rates (e.g., 19.2% for SigLIP in *Slight* violations with Long-term cases). These disparities arise from the distinct mechanisms of each MR: $MR_{condition}$ tightens task completion requirements (e.g., restricting FOV to 75%), and if fewer steps are used after mutation, it violates the monotonicity constraint (stricter conditions should not enable

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

more efficient solutions), effectively capturing such violations (e.g., 28.9% for SigLIP-Nav in Severe violations with Short-term cases); MR_{scene} selects the least similar scene for mutation, targeting behaviors that rely on irrelevant details, which violate its invariance constraint; $MR_{position}$ selects a new target maximally different from the original, and if the agent completes the task in fewer steps after the detour, it violates $C(\theta_{A\to B})+C(\theta_{B\to C}) \geq C(\theta_{A\to C})$, with the highest total detection rate among models reaching 67.2%; MR_{action} amplifies planners' vulnerabilities in action understanding and strategy adaptability by employing semantically conflicting actions and each of the four models has a total violation detection rate of over 20%. Thus, each MR targets distinct aspects of optimality, their combined performance attests to the method's comprehensiveness, and the lack of a single dominant MR shows they complement one another to cover diverse NoDs.

All four models exhibit the same severity-dependent trend. SigLIP-Det generally achieves the highest detection rates (e.g., 67.2% total detection rate for $MR_{position}$ in Short-term cases), while DINOv2 shows more moderate but still significant rates (e.g., 31.8% total detection rate for MR_{scene} in Short-term cases). This demonstrates that our MR-based method is robustly effective across different models, confirming its generality as a NoD detection framework. MRs target fundamental optimality properties of embodied task planning (invariance across positions, actions, conditions, and scenes) rather than model-specific behaviors. Even models with strong generalization capabilities (such as DINOv2) cannot avoid violations when undergoing NoD detection, ensuring the MRs remain effective. The higher detection rate of SigLIP-Det may stem from its focus on object detection and path localization, making it more sensitive to $MR_{position}$ and MR_{scene} further verifying that MRs can exploit model-specific vulnerabilities while maintaining broad applicability.

In summary, the experimental results confirm the effectiveness of the four MRs: they can stably detect NoDs across different *VS* categories, task lengths, and models, with performance tightly aligned to their design logic of targeting core optimality invariants via diversity-guided mutations.

2) Planning Quality Evaluation: In addition to the VDR, we report two standard navigation metrics to evaluate planning quality: *Success Rate (SR)*, which measures the fraction of episodes that successfully complete the task, and *Success weighted by Path Length (SPL)* [33], which jointly measures success and path efficiency relative to the expert trajectory. Since a NoD is defined only when the source plan succeeds, i.e., a successful source trajectory serves as the optimality reference, we compute the SR and SPL of the follow-up cases over the subset of episodes in which the source case succeeds. Hence, the source SR/SPL reflect each planner's overall navigation quality, while the follow-up SR/SPL are conditioned on source success, and we read each as a standalone indicator rather than a direct source-to-follow-up comparison.

Table II shows the comparison results. From the table, we can observe that the follow-up cases conditioned on source success maintain high success across all planners and MRs (SR $\geq$ 64% in every setting), indicating that most previously successful tasks remain successful after mutation. Therefore, the detected NoDs correspond to genuine optimality violations on successfully completed tasks, rather than mutation-induced failures. Among the four planners, SigLIP-Det achieves the best navigation quality, with a source SR/SPL of 84.5%/0.527 and follow-up averages of 87.7%/0.551, while simultaneously yielding the highest VDR (e.g., 50.3% for Position). The remaining planners (source SR/SPL of 48.3%/0.319 for DINOv2, 51.4%/0.355 for SigLIP, and 50.1%/0.330 for SigLIP-Nav) also maintain high success and improved SPL on the follow-up subset. This indicates that the high VDR of SigLIP-Det is not an artifact of weak navigation. On the contrary, even a strong navigator remains susceptible to NoDs, which demonstrates that NoD detection is complementary to conventional success-based evaluation.

TABLE II
SR (%), SPL, AND VDR (%) PER PLANNER. SOURCE COLUMNS ARE SHARED ACROSS ALL MRS; PER-MR COLUMNS REPORT FOLLOW-UP CASES OVER SOURCE-SUCCESS EPISODES.

Model	Source		Position			Action			Condition			Scene		
	SR	SPL	SR	SPL	VDR	SR	SPL	VDR	SR	SPL	VDR	SR	SPL	VDR
DINOv2	48.3	0.319	73.7	0.484	23.7	81.1	0.476	15.7	72.9	0.496	32.6	74.0	0.532	34.0
SigLIP	51.4	0.355	67.6	0.424	27.0	76.0	0.488	20.0	64.4	0.494	37.1	71.6	0.534	**40.6**
SigLIP-Nav	50.1	0.330	74.7	0.461	33.3	77.5	0.440	21.0	74.7	0.542	**40.7**	81.2	0.544	37.0
SigLIP-Det	**84.5**	**0.527**	**89.3**	**0.545**	**50.3**	**88.9**	**0.524**	**24.5**	**82.8**	**0.578**	36.9	**89.9**	**0.558**	36.3
Average	58.6	0.383	76.3	0.479	33.6	80.9	0.482	20.3	73.7	0.528	36.9	79.2	0.542	37.0

Furthermore, SR, SPL, and VDR characterize planning behavior at three progressively finer levels. SR captures whether the agent can *complete* the task; building on success, SPL captures how *efficient* the resulting path is relative to an expert reference; building on both, VDR captures whether the decision is *optimal*, i.e., self-consistent under metamorphic transformations, where a transformed variant should not admit a more efficient solution. Note that the first two are higher-is-better quality scores, whereas VDR is a diagnostic metric in which a higher value exposes more residual non-optimality. Hence, the three metrics are complementary: SR and SPL establish that a planner is a competent and efficient navigator, while VDR further reveals NoDs that persist even for such planners. This complementarity is reflected in our results. SigLIP-Det attains the highest SR/SPL yet still exhibits a high VDR, and the highest per-MR VDRs are also observed for SigLIP-Nav (Condition, 40.7%) and SigLIP (Scene, 40.6%), demonstrating that strong navigation quality and the presence of NoDs can coexist.

3) Violation Pattern Analysis: To gain a systematic and comprehensive understanding of the NoDs revealed by `NoD-DGMT`, we inspect MR violations and group them into four categories for in-depth exploration. Fig. 5 reports examples of issues detected by `NoD-DGMT`. These four examples respectively correspond to $MR_{position}$, MR_{action}, $MR_{condition}$, and MR_{scene} from left to right.

Inefficient Global Exploration Strategy. Fig. 5(a) shows the source path, and Fig. 5(b) shows the path with $MR_{position}$ applied. In Fig. 5(b), the agent navigates toward the farthest target from the original goal ("go to a pillow") for the first 1/4 of the path. This enables broader early exploration and more comprehensive environmental information acquisition. Upon switching back to the original goal at the 1/4 mark, the agent,

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

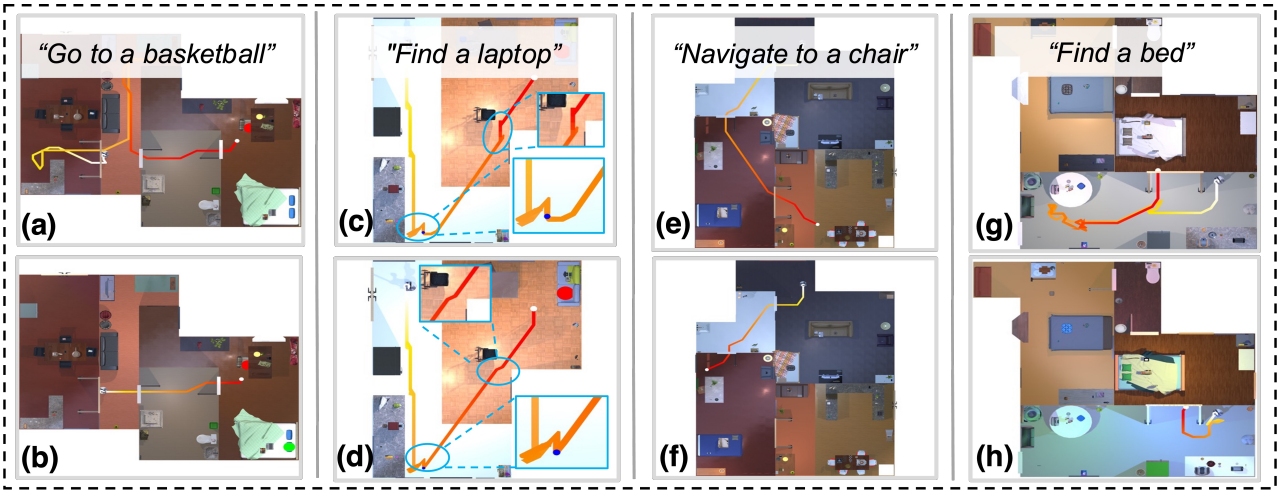


Fig. 5. Typical failure cases of NoD detected by NoD-DGMT.

leveraging prior global environmental awareness, plans a more direct, obstacle-avoiding path—thus skipping an extra room during exploration and shortening the route. In contrast, Fig. 5 (a), constrained by limited information, exhibits a detoured path.

Local Optimal Action Sequence Trapping. The second pattern involves agents becoming trapped in locally optimal action sequences. Fig. 5(c) shows the source path, and Fig. 5(d) shows the path with MR_{action} applied. In Fig. 5(d), we replace the original left-turn action with a right-turn. As shown, Fig. 5(d) eliminates a detour in the middle section compared to Fig. 5(c), resulting in a shorter route (over 40 steps fewer than the original result). In Fig. 5(c), the agent encountered a corner at the room edge, which acted as a distraction and led to a longer path. In Fig. 5(d), the reversed action altered the traveling direction, avoiding this distraction, and the replanned path led more directly to the target.

Inadequate Environmental Perception. Fig. 5(e) shows the source path, Fig. 5(f) is the path with $MR_{condition}$ applied, in which the agent's FOV reduced to 75%. In Fig. 5(f), the agent detects the chair at a closer distance, whereas in Fig. 5(e), it fails to detect the chair initially and continues exploring until a farther location. This is because with 75% FOV, the agent uses a more aggressive search strategy, prioritizing rapid exploration toward the target's general direction. Meanwhile, the smaller focus area increases the target chair's proportion in the sensor's FOV and enhances its contrast with the surroundings, enabling easier recognition, quicker positioning, and shorter path planning. In contrast, the algorithm of Fig. 5(e) tends to adopt a conservative global exploration strategy, requiring full environmental understanding before path planning—lengthening the exploration process.

Poor Environmental Adaptation. Fig. 5(g) shows the source path, while Fig. 5(h) presents the path after using MR_{scene}. It can be seen that the path length of Fig. 5(h) (44 steps) is much shorter than that of Fig. 5(g) (149 steps). Although the light in Fig. 5(h) is dimmer, the contrast between the bed and the surrounding environment is enhanced, making it easier for the agent to identify and locate the bed. In Fig. 5(g), the light is relatively strong, resulting in low distinguishability between the bed and the background, and the sensor imaging is blurred. Therefore, when the agent first enters the room with the bed, it fails to capture the bed, and the agent needs to spend more time searching and confirming the target position, which leads to a longer route.

The four MRs all reveal NoDs in embodied agents by introducing intentional changes to the original scenario, such as altering initial positions, reversing actions, adjusting sensory conditions, or modifying environmental settings. In each case, the modified scenario (via MR) results in a shorter, more efficient path than the original, exposing the agent's suboptimal planning in the source case. These inconsistencies arise because the agent fails to adapt effectively to environmental constraints, sensory limitations, or action choices, highlighting flaws in its decision-making logic when navigating toward targets.

C. RQ2: Efficiency of NoD-DGMT

We evaluate the efficiency of NoD-DGMT from two aspects: the per-test-case runtime overhead it adds on top of plain test execution, and how this overhead scales with task complexity.

1) Computational Overhead: Table III shows the per-test-case time breakdown of NoD-DGMT, measured with SigLIP on AI2-THOR. We decompose the per-case time into three parts: source-case execution, diversity-guided mutation generation and filtering, and follow-up execution. Since source execution is MR-independent, we report it once as an average (46.5 s). From the table, we can observe that the per-case time is dominated by simulator execution, i.e., the source and follow-up rollouts, which exceeds 90% of the total across all four MRs. This execution cost is inherent to any execution-based testing method, as it stems from rolling out test cases in the simulator rather than from NoD-DGMT itself. In contrast, the mutation generation and filtering introduced by NoD-DGMT adds only 2.7 to 8.8 s, accounting for 3.3% to 9.7% of the total. This cost is smallest for $MR_{condition}$ and MR_{action} and largest for MR_{scene}, where randomizing materials and lighting is more expensive to generate. Therefore, we can conclude that

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

TABLE III
PER-TEST-CASE COMPUTATIONAL OVERHEAD OF NoD-DGMT.

MR	Source	MR Gen. +Filter(s)	Follow-up (s)	Total (s)	Filter share
Position	46.5	3.1	31.9	81.5	3.8%
Action		2.8	34.5	83.8	3.3%
Condition		2.7	25.7	74.9	3.6%
Scene		8.8	35.4	90.7	9.7%

TABLE IV
SCALABILITY OF NoD-DGMT WITH TASK COMPLEXITY.

Complexity	Source (s)	MR Gen. +Filter (s)	Follow-up (s)	Total (s)	Filter share
Short	36.1	4.1	19.3	59.5	6.9%
Medium	41.3	4.1	30.5	75.8	5.4%
Long	76.7	5.1	55.7	137.5	3.7%

the overhead of NoD-DGMT over plain execution is negligible relative to the unavoidable simulator cost. Although measured with a single planner, this conclusion is planner-agnostic: the generation and filtering cost does not depend on the planner, while the dominant simulator cost applies to any planner.

2) Scalability with Task Complexity: To assess whether NoD-DGMT remains practical as tasks grow larger, we analyze how its per-case time scales with task complexity, using the Short, Medium, and Long categories defined by reference path length (Table IV). From the table, we can observe that the total time per case grows from 59.5 s on Short tasks to 137.5 s on Long tasks, a $2.3\times$ increase that is driven almost entirely by simulator execution, since longer tasks require more steps to roll out. In contrast, the cost of mutation generation and filtering remains nearly constant (4.1 to 5.1 s), as it is largely independent of trajectory length. Consequently, its relative overhead *decreases* with task complexity, from 6.9% on Short tasks to 3.7% on Long tasks. Therefore, we can conclude that NoD-DGMT scales favorably, with an overhead that becomes increasingly negligible as tasks grow more complex, confirming its feasibility for practical deployment.

D. RQ3: Comparison with Other Testing Methods

To demonstrate the advantages of NoD-DGMT, we compare it with seven baseline methods: two Random Testing (RT) methods, two Property-Based Testing (PBT) methods, and three MT methods. We choose these families for their methodological diversity, reproducibility under our experimental setup, and representativeness of established or state-of-the-art approaches. RT generates test cases without guidance, serving as a lower-bound reference. PBT checks hand-specified properties that an optimal plan should satisfy, and thus depends on the strength of the encoded property. MT, the paradigm most closely related to ours, derives oracles from relations between source and follow-up cases.

The two RT methods generate perturbed model weights to search for shorter paths: RT-GF employs Gaussian Fuzzing, while RT-NAI utilizes Neuron Activation Inverse [34]. For each test case, a NoD is identified if the perturbed model produces a shorter path than the original one. The two PBT methods exploit specific properties of optimal planning. PBT-NR (No Revisit) leverages the principle that an optimal planner should not revisit the same spatial grid more than twice. PBT-SP (Sub-Problems) initializes the agent at intermediate positions along the original trajectory, exploiting the property that a sub-problem path should not exceed the corresponding segment of the full-problem solution. The three MT methods adapt metamorphic relations to embodied navigation. MT-Obstacle, adapted from MRIP2 [35], introduces additional obstacles to test planning robustness. MT-Transform, adapted from [36], applies three transformations (Points-related, Threat-environments-related, and Geometric) to evaluate planning consistency across environmental variations. MT-Synonym, adapted from a recent VLA-specific relation [37], replaces the action verb with a meaning-preserving synonym, under the invariance assumption that semantically equivalent instructions should yield consistent plans.

Table V presents a comparative analysis of the NoD detection performance between our proposed MRs-based method (encompassing four MRs: position, action, condition, and scene) and seven baseline testing approaches. The results demonstrate the superiority of our MR-based method. Across all models, our method achieves strong detection rates with average performances of 33.6% (Position), 20.3% (Action), 36.9% (Condition), and 37.0% (Scene), respectively. In contrast, the seven baseline methods exhibit significantly lower detection rates: the average rates for MT-Obstacle, MT-Transform, MT-Synonym, PBT-SP, PBT-NR, RT-GF, and RT-NAI are 20.3%, 27.3%, 19.0%, 12.2%, 5.1%, 2.2%, and 3.2%, respectively. Overall, our four well-designed MRs achieve an average violation detection rate of 31.9%, which is substantially higher than that of the baseline methods.

Among our four MRs, Scene and Condition achieve the highest detection rates (37.0% and 36.9%), indicating that environmental and perceptual variations are particularly effective at exposing planning inefficiencies. Position MR shows strong performance (33.6%), especially on SigLIP-Det (50.3%), which benefits from ground truth detection that enhances sensitivity to spatial relationships. Action MR exhibits moderate performance (20.3%), as action-level perturbations require models with sophisticated long-horizon planning capabilities to reveal non-optimality.

Across the four models evaluated, SigLIP-Det achieves the highest overall detection rates, particularly for Position MR (50.3%). Its ground truth detection capability makes it more sensitive to spatial perturbations, facilitating NoD detection. SigLIP-Nav shows strong performance on Condition MR (40.7%), benefiting from its extended training on navigation tasks. SigLIP demonstrates high detection rates for Scene MR (40.6%), indicating sensitivity to environmental variations. DINOv2 exhibits relatively lower detection rates across all MRs, likely due to stronger generalization capabilities that maintain robust performance despite environmental perturbations.

Among the baselines, MT-Transform (27.3%) performs best, as its transformation-based approach shares conceptual similarities with our MR design. However, it lacks the systematic diversity-guided selection, resulting in lower effectiveness. MT-Obstacle (20.3%) shows moderate performance but is limited to obstacle-related scenarios. MT-Synonym (19.0%) leaves the underlying planning problem unchanged, so it

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

TABLE V
NOD VIOLATION DETECTION PERFORMANCE COMPARISON WITH DIFFERENT BASELINE METHODS.

Testing Method	NoD-DGMT				Baseline Methods						
					Metamorphic Testing			Property-Based Testing		Random Testing	
Model	Position	Action	Condition	Scene	MT-Obs.	MT-Trans.	MT-Syn.	PBT-SP	PBT-NR	RT-GF	RT-NAI
DINOv2	23.7%	15.7%	32.6%	34.0%	14.1%	19.8%	18.3%	8.6%	3.33%	1.1%	0.0%
SigLIP	27.0%	20.0%	37.1%	**40.6%**	17.5%	26.4%	13.8%	12.0%	3.81%	0.0%	0.0%
SigLIP-Nav	33.3%	21.0%	**40.7%**	37.0%	**25.2%**	28.9%	21.4%	**14.3%**	6.67%	0.0%	0.0%
SigLIP-Det	**50.3%**	**24.5%**	36.9%	36.3%	24.3%	**33.9%**	**22.6%**	13.8%	**6.75%**	**7.8%**	**12.7%**
Average	33.6%	20.3%	36.9%	37.0%	20.3%	27.3%	19.0%	12.2%	5.1%	2.2%	3.2%
	31.9%										

TABLE VI
PER-TEST-CASE TIME (SECONDS) COMPARISON WITH DIFFERENT BASELINE METHODS.

Metric	NoD-DGMT	MT-Obs.	MT-Trans.	MT-Syn.	PBT-SP	PBT-NR	RT-NAI	RT-GF
Total (s)	**82.7**	77.0	83.3	71.8	65.4	46.5	218.1	163.7
MR time (s)	**4.4**	3.4	5.0	2.8	2.7	0.0	2.7	2.7
Share (%)	**5.3**	4.4	6.0	3.9	4.1	0.0	1.2	1.6

TABLE VII
NOD DETECTION RATE (%) UNDER SIMULATED SENSOR NOISE AND ACTUATOR LATENCY. SEVERITY 0 DENOTES THE CLEAN SETTING.

Perturbation	Severity					
	L0	L1	L2	L3	L4	L5
Speckle noise	31.9	27.5	25.9	24.1	22.8	21.4
Actuator latency	31.9	31.3	21.7	17.5	16.7	11.2

exposes a NoD only when the planner is sensitive to surface wording. The detection rates rise monotonically with perturbation strength, and NoD-DGMT extends this trend: its mutations actively drive the agent into new states and observations that are constructed to elicit more efficient alternatives, yielding the highest detection rate (31.9%).

The property-based and random methods perform clearly worse than the MT baselines, as they rely on weaker oracles. Among them, PBT-SP (12.2%) outperforms PBT-NR (5.1%), since sub-problem decomposition aligns better with hierarchical planning and particularly benefits SigLIP-Nav (14.3%), whereas PBT-NR's single hand-specified rule captures only a narrow set of NoD patterns. The RT methods perform worst (RT-GF 2.2%, RT-NAI 3.2%), as undirected weight perturbations rarely produce shorter paths. Therefore, we can conclude that NoD-DGMT outperforms every baseline, confirming that systematic, optimality-targeted metamorphic testing is substantially more effective for NoD detection than alternative strategies.

Table VI shows per-test-case time of NoD-DGMT and the baseline methods, measured with SigLIP on AI2-THOR. Compared with the baselines, NoD-DGMT remains competitive in cost. Its per-case time (74.9 to 90.7 s) is on par with the metamorphic-testing baselines (71.8 to 83.3 s) and the property-based PBT-SP (65.4 s), and far below the random-testing methods RT-NAI (218.1 s) and RT-GF (163.7 s), whose weight-perturbation follow-ups are markedly more expensive. PBT-NR performs no follow-up generation, as its no-revisit rule is checked on the source plan alone, so it incurs no extra cost. Overall, NoD-DGMT attains a substantially higher detection rate at a cost comparable to the cheapest baselines, confirming that its diversity-guided generation is efficient.

E. RQ4: Robustness of NoD-DGMT

In real-world deployment, embodied agents often operate under non-idealized perception and actuation, such as sensor noise and actuator delay, whereas AI2-THOR provides clean observations and precise, low-latency control. To assess the robustness of NoD detection under such conditions, we introduce two perturbation models and measure the detection rate with four models at five increasing severity levels each (Severity 0 denotes the clean setting used in our main experiments). For perception, inspired by standard corruption-robustness benchmarks such as ImageNet-C [38] and RobustNav [20], we apply multiplicative *speckle noise* to camera observations at five increasing levels spanning mild to severe corruption ($c \in \{0.15, 0.20, 0.35, 0.45, 0.60\}$). For actuation, following prior analyses that identify actuation delay as a key real-world non-ideality [39], we inject an *actuator delay* of 1 to 5 steps. The clean detection rate at Severity 0 (31.9%) is the average of four models over the four MRs.

Table VII reports the results. Under sensor noise, detection decreases gradually as severity increases, dropping from 31.9% in the clean setting to 25.9% at Severity 2 and 21.4% at Severity 5. This retains about 67% of the clean detection rate at the highest severity, suggesting that perception noise weakens but does not eliminate the ability of NoD-DGMT to expose NoDs. Under actuator latency, detection is more sensitive: it remains close to the clean setting at Severity 1 (31.3%), but drops sharply from Severity 2 onward and reaches 11.2% at Severity 5, about 35% of the clean baseline. This larger degradation is expected because high actuator latency introduces execution uncertainty, causing realized trajectories to deviate from the planner's intended actions. As a result, the source–follow-up cost difference becomes less attributable to the MR transformation alone, weakening the signal used to expose NoDs. Overall, NoD-DGMT retains a non-trivial detection capability under non-idealized perception and actuation, while its effectiveness decreases as execution uncertainty increases, especially under severe actuator latency.

F. RQ5: Generalization of NoD-DGMT

The previous experiments are all conducted in AI2-THOR, an indoor simulator with discrete actions and idealized perception. To assess whether NoD-DGMT generalizes beyond this setting, we apply it in a substantially different environment: MetaUrban [40], an *outdoor* urban-navigation simulator with

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

TABLE VIII
GENERALIZATION EVALUATION ON THE OUTDOOR EMBODIED SIMULATOR METAURBAN.

Method	Source		Position			Action			Condition			Scene		
	SR	SPL	SR	SPL	VDR	SR	SPL	VDR	SR	SPL	VDR	SR	SPL	VDR
PPO	43.1	0.421	66.7	0.660	**28.6**	91.3	0.896	15.2	**79.5**	0.774	**30.8**	82.5	0.807	30.0
PPO-ET	51.4	0.514	73.1	0.731	7.7	**98.0**	**0.980**	10.2	70.0	0.700	18.0	81.4	0.811	30.2
PPO-Lag	**57.7**	**0.576**	**85.2**	**0.845**	14.8	80.0	0.798	**16.7**	78.2	**0.781**	25.5	**85.7**	**0.856**	**40.8**
Average	50.7	0.504	75.0	0.745	17.0	89.8	0.891	14.0	75.9	0.752	24.8	83.2	0.825	33.7

continuous control and *LiDAR*-based perception, where the agent is a delivery robot operating among crosswalks, pedestrians, and roadside obstacles. This shift, from indoor discrete ObjectNav to outdoor continuous urban navigation, stresses the generality of our MRs across simulators, action spaces, and sensing modalities. As MetaUrban is a safety-critical domain, we also use it to examine whether NoD-DGMT remains effective when navigation quality is governed by stronger, safety-oriented objectives.

We evaluate three RL planners: standard Proximal Policy Optimization (PPO) [41], which maximizes task reward, and two Safe Reinforcement Learning (SafeRL) variants that additionally enforce safety constraints, namely PPO-ET, which recasts the constrained Markov Decision Process (MDP) as an early-terminated MDP [42], and PPO-Lag, which adds a Lagrangian constraint term to the objective [43]. We adopt the Point Navigation (PointNav) task: each policy is trained on a subset of MetaUrban-train with the provided hyperparameters [40] (12 GPU-hours on a single V100) and evaluated on 100 unseen MetaUrban scenes. PPO is implemented with Stable-Baselines3 [44] and the SafeRL variants with OmniSafe [45].

Table VIII reports SR, SPL, and VDR for each planner across the four MRs. From the table, we can observe that all four MRs remain effective in this outdoor, continuous-control domain. The average VDRs are 17.0% ($MR_{position}$), 14.0% (MR_{action}), 24.8% ($MR_{condition}$), and 33.7% (MR_{scene}), showing that NoD-DGMT can expose NoDs beyond the indoor ObjectNav setting used in our main evaluation. Among the four, MR_{scene} is the most effective, reaching VDRs of 30.0%, 30.2%, and 40.8% on PPO, PPO-ET, and PPO-Lag, respectively. Notably, PPO-Lag attains the best source navigation quality (57.7% SR, 0.576 SPL) yet still exhibits the highest Scene VDR (40.8%). This indicates that explicit safety constraints improve navigation quality but do not eliminate NoDs, which our MRs continue to surface. Therefore, NoD-DGMT transfers across simulators, action spaces, and sensing modalities, and remains effective even under safety-oriented objectives. Notably, all four MRs here encode *generic* optimality, yet they remain effective in this new domain without any task-specific adaptation, underscoring the broad applicability of NoD-DGMT.

To illustrate a concrete NoD that NoD-DGMT surfaces in this new outdoor domain, Fig. 6 presents a case from the PPO planner in MetaUrban. Figs. 6(a) and (c) show top-down views of the source and the follow-up paths, both from the same origin (yellow) to the same goal (red), while (b) is a third-person chase-camera view during the source rollout. In the source case, the delivery robot crosses the intersection and then makes a pronounced rightward detour, wandering through the roadside area past the post box and telephone booth before reaching the goal, whereas after the transformation it follows a more direct route through the same crosswalk along a noticeably shorter path. Since both trajectories share the same origin, goal, and road layout, the shorter follow-up path proves that the source plan is non-optimal. Notably, this detour is not a failure: the robot completes the task successfully, yet the unnecessary roaming lengthens the route and prolongs exposure to crosswalks and obstacles, costs that are invisible to functional metrics but accumulate over repeated deliveries. This is precisely the kind of NoD that NoD-DGMT exposes, demonstrating its practical value even in this outdoor, safety-critical setting.



Fig. 6. A non-optimal decision of the PPO planner detected by NoD-DGMT in MetaUrban. (a) the source path with an over-conservative detour, (b) a third-person chase-camera view captured during the detour, and (c) the shorter follow-up path that exposes the source plan as non-optimal.

G. RQ6: Usefulness of the Diversity-Guided Filter

To measure the diversity of violation cases, we apply spectral clustering [46], using Euclidean distance for the initial views and Levenshtein distance for the action sequences, and determine the optimal number of clusters by maximizing the Silhouette Coefficient [47]. The overall *Violation Diversity (VD)* is then defined as the number of unique cluster pairs $(c_{\text{view}}, c_{\text{action}})$ across all cases, where c_{view} and c_{action} denote the cluster assignments based on the initial view and the action sequence, respectively:

$$VD = \left|\left\{(c^i_{\text{view}}, c^i_{\text{action}}) \mid i = 1, 2, \ldots, N\right\}\right|, \tag{5}$$

where N is the total number of violation cases and $|\cdot|$ denotes set cardinality.

Table IX compares the violation detection rates and diversity scores of NoD-DGMT with and without the diversity-guided filter across models and MRs. For $MR_{position}$, the filter improves the detection rate by 4.3%, 0.9%, 6.9%, and 6.7% for DINOv2, SigLIP, SigLIP-Nav, and SigLIP-Det, respectively. The gain is minimal on SigLIP (0.9%) but substantial on SigLIP-Nav and SigLIP-Det (6.9% and 6.7%). We attribute this to two factors: SigLIP-Nav's extended navigation training improves object recognition, and SigLIP-Det's ground-truth detector increases sensitivity to target-position changes, enabling more precise mutation and removing noise from inaccurate localization. Consequently, position mutation achieves both higher detection rates (50.3% for SigLIP-Det, 33.3% for SigLIP-Nav) and stronger filter effectiveness on these models. Interestingly, SigLIP shows the smallest detection-rate gain yet the largest diversity gain (20 to 24 clusters), suggesting that weak localization, though noisier, also generates more novel

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

TABLE IX
VIOLATION DETECTION RATE COMPARISON OF NoD-DGMT WITH AND WITHOUT THE DIVERSITY-GUIDED FILTER

Metric	Model	Position			Action			Condition			Scene			Average		
		Filter	w/o	Enhance	Filter	w/o	Enhance	Filter	w/o	Enhance	Filter	w/o	Enhance	Filter	w/o	Enhance
Violation	DINOv2	23.7%	19.4%	4.3%	15.7%	13.7%	2.0%	32.6%	28.4%	4.2%	34.0%	25.8%	**8.2%**	26.5%	21.8%	4.7%
	SigLIP	27.0%	26.1%	0.9%	20.0%	19.0%	1.0%	37.1%	29.5%	**7.6%**	**40.6%**	35.6%	5.0%	31.2%	27.6%	3.6%
	SigLIP-Nav	33.3%	26.5%	**6.9%**	21.0%	18.1%	2.9%	**40.7%**	36.1%	4.6%	37.0%	35.0%	2.0%	33.0%	28.9%	4.1%
	SigLIP-Det	**50.3%**	43.6%	6.7%	**24.5%**	19.0%	**5.5%**	36.9%	32.1%	4.8%	36.3%	33.9%	2.4%	**37.0%**	32.2%	**4.8%**
Diversity	DINOv2	16	14	2	14	10	**4**	18	16	2	18	16	2	16.5	14	2.5
	SigLIP	24	20	**4**	14	13	1	24	18	**6**	21	18	3	20.75	17.25	3.5
	SigLIP-Nav	25	22	3	16	12	**4**	24	18	**6**	**23**	18	**5**	22	17.5	**4.5**
	SigLIP-Det	**35**	33	2	**19**	15	**4**	**30**	25	5	22	22	0	**26.5**	23.75	2.75

trajectories. This inverse relationship indicates that planners with stronger localization discover violations of both higher quantity and higher quality.

Compared with the other mutations, MR_{action} applies the shortest and most conservative perturbation, with filtered detection rates of 15.7%, 20.0%, 21.0%, and 24.5% across the four models. The filter yields the largest gain on SigLIP-Det (5.5%), indicating that extraneous action perturbations affect higher-performing planners more strongly: a planner with stronger long-horizon planning reacts more sensitively to a perturbed action and re-plans accordingly, so a stronger perturbation more readily exposes its non-optimality. For the same reason, semantically distant actions induce more varied subsequent behaviors than random ones, so the diversity gain of MR_{action} follows the same trend as its detection-rate gain.

For $MR_{condition}$, the filter yields the largest detection-rate gain on SigLIP (37.1%, +7.6%) and the smallest on DINOv2 (32.6%, +4.2%), and the diversity gains follow the same order (6 vs. 2 clusters). Since SigLIP's image encoder outperforms DINOv2 on the CHORES-$\mathbb{S}$ test set, this indicates that higher-performing encoders are more sensitive to receptive-field reduction: a tighter field of view prompts them to re-examine the scene and occasionally find a more direct path that the original plan had missed. Balancing planner performance against detection efficiency thus remains an open question.

MR_{scene} targets a planner's robustness to task-irrelevant scene variations. The detection rate on SigLIP (40.6%) exceeds that on DINOv2 (34.0%), indicating that although DINOv2 is weaker on the task, it generalizes more consistently across scene variations for the same task. The only exception is the diversity gain on SigLIP-Det (0 clusters), which we attribute to its tolerance to scene variations.

The diversity-guided filter is consistently effective across all models and MRs. From Table IX, it improves the detection rate by an average of 4.7%, 3.6%, 4.1%, and 4.8% for DINOv2, SigLIP, SigLIP-Nav, and SigLIP-Det, respectively, while improving the diversity score by 2.5, 3.5, 4.5, and 2.75 clusters. This dual gain matters because it addresses both the quantity (detection efficiency) and the quality (coverage breadth) of NoD detection, and the consistent improvements across architectures validate the generalizability of our selection strategy.

The enhancement patterns also vary informatively across MRs. Planners with stronger perception (e.g., accurate localization, better image encoders) improve more under position and condition mutations, while those with stronger planning benefit more from action mutations. These findings show that the diversity-guided filter not only improves detection comprehensiveness but also reveals model-specific strengths and weaknesses.

H. RQ7: Justification of Key Configurations

Each MR involves a key configuration choice. We ablate these choices to analyze their impact on NoD detection and to justify the settings adopted by default. Table X reports the violation detection rate of each configuration across the four planners. For $MR_{position}$, we vary the temporary-target step fraction $\rho \in \{0.1, 0.25, 0.5\}$. For MR_{action}, we vary the selector that determines the semantically most divergent substitute action. For $MR_{condition}$, we vary the field-of-view scaling factor FOV $\in \{0.5, 0.75, 0.9\}$. For MR_{scene}, we vary the randomization scope (materials only, lighting only, or both).

For $MR_{position}$, a moderate ρ=0.25 yields the highest average detection rate (33.6%), close to ρ=0.1 (32.9%) but well above ρ=0.5 (20.7%), as an overly long detour deviates too far from the original task. For $MR_{condition}$, an intermediate FOV=0.75 is the most effective (36.9%), outperforming both 0.9 (30.4%) and 0.5 (24.5%), since an excessive reduction degrades baseline perception rather than merely tightening the optimality condition. For MR_{scene}, randomizing both materials and lighting (37.0%) outperforms randomizing either alone (materials 30.4%, lighting 31.2%), as a larger visual change more strongly stresses robustness to task-irrelevant variations. For MR_{action}, the LLM-based selector (GPT-4) attains the highest average rate (20.3%), above the embedding-based CLIP [48] (16.4%) and SBERT [49] (16.3%), and is best on three of the four planners, with SigLIP-Nav the only exception; the embedding selectors thus remain a viable, lower-cost alternative when the cost, latency, and non-determinism of an LLM (Section VI) are a concern. Based on these results, we adopt the strongest setting of each MR (ρ=0.25, FOV=0.75, Mat. & Light, and the LLM-based selector) in our main experiments.

The violations detected above also let us examine the empirical distribution of VS and verify the thresholds used to classify them. As shown in Fig. 7, the 10% and 20% cutoffs align well with the distribution of VS over the 616 detected violations, approximating its first quartile ($\approx$ 8.8%) and median ($\approx$ 21.5%), respectively. This produces three well-populated levels (Slight 28.7%, Moderate 19.3%, Severe 51.9%) and gives each threshold a clear meaning: Slight

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

TABLE X
JUSTIFICATION OF PER-MR CONFIGURATIONS.

Model	**Position** (ρ)			**Action** (selector)			**Condition** (FOV)			**Scene** (scope)		
	0.1	0.25	0.5	CLIP	SBERT	LLM	0.5	0.75	0.9	Material	Light	Mat. & Light
DINOv2	25.3	23.7	14.9	12.5	13.2	15.7	20.2	32.6	26.9	29.2	32.6	34.0
SigLIP	38.5	27.0	15.0	14.2	14.4	20.0	17.5	37.1	26.5	34.0	36.1	40.6
SigLIP-Det	40.4	50.3	28.0	16.5	16.1	24.5	37.3	36.9	33.1	29.7	31.0	36.3
SigLIP-Nav	27.5	33.3	24.8	22.5	21.4	21.0	23.2	40.7	35.3	28.9	25.3	37.0
Average	32.9	**33.6**	20.7	16.4	16.3	**20.3**	24.5	**36.9**	30.4	30.4	31.2	**37.0**

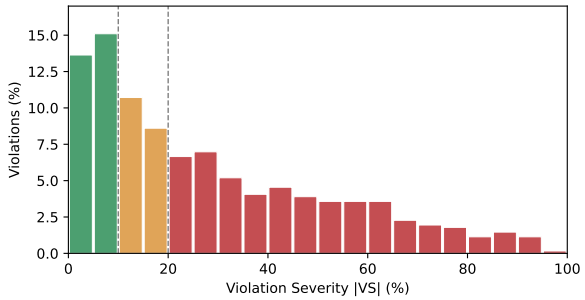


Fig. 7. Empirical distribution of Violation Severity (VS) over the 616 detected violations. Dashed lines mark the 10% and 20% thresholds separating Slight, Moderate, and Severe violations.

collects the least-severe deviations, whereas Severe captures the more inefficient upper half of violations, whose heavy tail extends beyond 90%.

VI. THREATS TO VALIDITY

Construct validity. The main threat is whether our optimality checker faithfully reflects planning quality. First, NoD-DGMT reduces plan quality to step count. In embodied task planning, where a plan is a sequence of discrete primitive actions of roughly constant duration and energy, step count is a natural cost, proportional to execution time and a proxy for actuation energy. Other tasks may need richer costs, such as path length or energy for continuous navigation, which our configurable cost function can accommodate without changing the detection pipeline. Second, the severity thresholds are configurable; to justify our defaults, we ground them in the empirical distribution over the 616 detected violations rather than fixing them a priori, and report SR and SPL alongside the detection rate so that no single measure drives our conclusions. Third, our MRs characterize relative rather than global non-optimality, a deliberate trade-off of the oracle-free design that lets NoD-DGMT scale without any ground-truth optimal plan, with detection conditioned on a successful source plan to match our definition of a NoD. A detected violation is sound because the more efficient follow-up plan is itself feasible for the source task, so it witnesses a genuinely superior alternative.

Internal validity. The main threats concern factors in the experimental setup that may affect the results. First, differences in the experimental setup across methods could bias the comparison. To ensure a fair comparison, all baselines are execution-based and run under the same simulator, planners, and task suite, so that NoD-DGMT's advantage is attributable to the method rather than the setup. Second, the semantic action selection relies on an external LLM, which could raise reproducibility concerns. To mitigate this, we query GPT-4 with the decoding temperature set to 0 for deterministic selection, and our ablation shows that deterministic embedding-based selectors such as CLIP and SBERT reach detection rates within a few points of the LLM, so our findings do not hinge on a specific model. Third, the diversity-guided filter keeps only the single most-divergent follow-up per source case, a deliberate trade-off between coverage and efficiency: exhaustively testing all candidate follow-ups would be more thorough but far more costly. Because some violations exposed by other follow-ups may be missed, our reported detection rates should be read as a conservative lower bound rather than an overestimate.

External validity. The main threat to external validity is the generalizability of our results. First, our evaluation targets embodied task planning and a finite set of planners and datasets. To mitigate this, we evaluate four representative planners across multiple datasets, and we further validate on a second, outdoor embodied simulator, MetaUrban, where all four MRs remain effective and even safety-constrained SafeRL planners exhibit non-optimal decisions, showing that the generic MRs transfer to a substantially different outdoor domain. Second, the four MRs are intentionally designed to capture generic optimality rather than domain-specific criteria. While this could be seen as a limitation, it is a deliberate choice that ensures broad applicability across embodied tasks without task-specific tuning, and the framework lets users add domain-specific costs or constraints as new MRs without changing the detection pipeline. Third, our evaluation is conducted in simulation, which may not fully reflect real-world perception and actuation. To partially mitigate this, we evaluate NoD detection under injected sensor noise and actuator latency at five severity levels across two simulators. In the future, we will validate NoD-DGMT on physical robots and hardware-in-the-loop platforms to further assess its effectiveness under real-world conditions.

VII. RELATED WORK

Embodied Agent Evaluation. Existing research on embodied agent verification primarily focuses on functional correctness and domain specifications. Formal verification methods such as model checking mainly target safety-critical properties but rarely address non-functional attributes like plan cost or execution efficiency [50] [51] [52]. Moreover, existing evaluation work is severely insufficient when it comes to non-functional requirements, particularly the agents' ability to make optimal decisions and generate optimal paths [53]. Current methods for testing AI planners primarily focus on two aspects: assessing the extent to which these planners comply with the planning domain definition language [54] [55], and validating their input domains and problem definitions [56]. However, despite ongoing efforts to formally verify the correctness of generated plans [57] and test domain/problem definitions [56], a significant challenge remains in determining whether the generated plans are optimal. Currently, there is no systematic evaluation method for the optimality of embodied agents [58]. Existing research has not solved core problems such as "how to define optimality", "how to generate scenarios that can expose NoDs", and "how to verify whether a decision is optimal".

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

Optimality in Embodied Agents. While some work pursues specific forms of optimality (e.g., shortest trajectories in path planning), systematic evaluation frameworks remain lacking. MT has been applied to action policies in gaming and planning contexts [35], [53], employing state relaxation techniques to formulate MRs. However, these applications lack specific focus on optimality evaluation in embodied agents. Zhang et al. [36] utilized triangle inequality-based MRs to test graph-based path-planning algorithms, but this approach is limited to specific path-planning scenarios and lacks generalizability to broader embodied agent domains. Cheng et al. [58] employed MT to evaluate learned policies in autonomous driving, where input scenarios are mutated to preserve optimality properties. While sharing conceptual similarities with our approach in MR design, their work focuses on policy evaluation in constrained road driving scenarios rather than general NoD detection across diverse embodied agent tasks. Mazouni et al. [59] integrated mutation testing with MT for AI planner optimality, introducing a mutation adequacy-based state selection strategy. Valle et al. [37] used execution-level MRs to evaluate VLA-enabled manipulation robots, focusing on behavioral consistency. However, these existing approaches remain limited to specific domains and lack the diversity-guided test case selection mechanism necessary for comprehensive NoD detection across different violation types.

Our work addresses these limitations by proposing a systematic framework for NoD detection in embodied agents through diversity-guided metamorphic testing. Unlike prior work that focuses on domain-specific optimality or policy evaluation, we provide a comprehensive approach that combines carefully designed metamorphic relations with diversity-guided selection to detect diverse types of planning inefficiencies across different embodied agent architectures.

VIII. Conclusion

In this paper, we propose `NoD-DGMT`, the first MT framework designed to evaluate the decision optimality of embodied agents. We formalize the NoDs problem and define four specialized metamorphic relations capturing fundamental optimality properties. Our approach systematically detects optimality violations through diversity-guided metamorphic transformations without requiring ground-truth optimal solutions, enabling comprehensive assessment of decision optimality across diverse planning scenarios. The experimental results demonstrate the effectiveness, efficiency, robustness, and generalizability of NoD-DGMT across multiple planners and simulation environments. Our work addresses a critical gap in embodied AI evaluation and provides a systematic methodology that will guide the development of more efficient and reliable embodied agents for future real-world applications. To ensure reproducibility, our artifact and experimental results are publicly available at https://nod-dgmt.github.io/project/.

References

[1] Y. Yang, T. Zhou, K. Li, D. Tao, L. Li, L. Shen, X. He, J. Jiang, and Y. Shi, "Embodied multi-modal agent trained by an llm from a parallel textworld," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 26 275–26 285.

[2] H. Liu, D. Guo, and A. Cangelosi, "Embodied intelligence: A synergy of morphology, action, perception and learning," *ACM Computing Surveys*, 2025.

[3] R. Mon-Williams, G. Li, R. Long, W. Du, and C. G. Lucas, "Embodied large language models enable robots to complete complex tasks in unpredictable environments," *Nature Machine Intelligence*, vol. 7, no. 4, pp. 592–601, 2025.

[4] Z. Feng, R. Xue, L. Yuan, Y. Yu, N. Ding, M. Liu, B. Gao, J. Sun, X. Zheng, and G. Wang, "Multi-agent embodied ai: Advances and future directions," *Science China Information Sciences*, 2025.

[5] Y. Ge, Y. Tang, J. Xu, C. Gokmen, C. Li, W. Ai, B. J. Martinez, A. Aydin, M. Anvari, A. K. Chakravarthy *et al.*, "Behavior vision suite: Customizable dataset generation via simulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 22 401–22 412.

[6] H. Fan, X. Liu, J. Y. H. Fuh, W. F. Lu, and B. Li, "Embodied intelligence in manufacturing: leveraging large language models for autonomous industrial robotics," *Journal of Intelligent Manufacturing*, vol. 36, no. 2, pp. 1141–1157, 2025.

[7] Y. Liu, X. Cao, T. Chen, Y. Jiang, J. You, M. Wu, X. Wang, M. Feng, Y. Jin, and J. Chen, "From screens to scenes: A survey of embodied ai in healthcare," *Information Fusion*, vol. 119, p. 103033, 2025.

[8] Y. Zhou, L. Huang, Q. Bu, J. Zeng, T. Li, H. Qiu, H. Zhu, M. Guo, Y. Qiao, and H. Li, "Embodied understanding of driving scenarios," in *European Conference on Computer Vision*. Springer, 2024, pp. 129–148.

[9] Y. Huang, J. Sansom, Z. Ma, F. Gervits, and J. Chai, "Drivlme: Enhancing llm-based autonomous driving agents with embodied and social experiences," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 3153–3160.

[10] L. Zhang, Y. Wang, H. Gu, A. Hamidizadeh, Z. Zhang, Y. Liu, Y. Wang, D. G. A. Bravo, J. Dong, S. Zhou *et al.*, "Et-plan-bench: Embodied task-level planning benchmark towards spatial-temporal cognition with foundation models," *arXiv preprint arXiv:2410.14682*, 2024.

[11] M. Chang, G. Chhablani, A. Clegg, M. D. Cote, R. Desai, M. Hlavac, V. Karashchuk, J. Krantz, R. Mottaghi, P. Parashar *et al.*, "Partnr: A benchmark for planning and reasoning in embodied multi-agent tasks," in *International Conference on Learning Representations (ICLR)*, 2025.

[12] S. Wang, Z. Fei, Q. Cheng, S. Zhang, P. Cai, J. Fu, and X. Qiu, "World modeling makes a better planner: Dual preference optimization for embodied task planning," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[13] J.-W. Choi, Y. Yoon, H. Ong, J. Kim, and M. Jang, "Lota-bench: Benchmarking language-oriented task planners for embodied agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] M. Lei, Y. Zhao, G. Wang, Z. Mai, S. Cui, Y. Han, and J. Ren, "Stma: A spatio-temporal memory agent for long-horizon embodied task planning," *arXiv preprint arXiv:2502.10177*, 2025.

[15] M. Li, S. Zhao, Q. Wang, K. Wang, Y. Zhou, S. Srivastava, C. Gokmen, T. Lee, E. L. Li, R. Zhang *et al.*, "Embodied agent interface: Benchmarking llms for embodied decision making," *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 428–100 534, 2024.

[16] K. Ehsani, T. Gupta, R. Hendrix, J. Salvador, L. Weihs, K.-H. Zeng, K. P. Singh, Y. Kim, W. Han, A. Herrasti *et al.*, "Spoc: Imitating shortest paths in simulation enables effective navigation and manipulation in the real world," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 16 238–16 250.

[17] J. Hu, R. Hendrix, A. Farhadi, A. Kembhavi, R. Martín-Martín, P. Stone, K.-H. Zeng, and K. Ehsani, "Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

[18] X. Zhang, H. Qin, F. Wang, Y. Dong, and J. Li, "Lamma-p: Generalizable multi-agent long-horizon task allocation and planning with lm-driven pddl planner," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

[19] Y. Zhong, C. Feng, F. Yan, F. Liu, L. Zheng, and L. Ma, "P3nav: A unified framework for embodied navigation integrating perception, planning, and prediction," *arXiv preprint arXiv:2503.18525*, 2025.

[20] P. Chattopadhyay, J. Hoffman, R. Mottaghi, and A. Kembhavi, "Robustnav: Towards benchmarking robustness in embodied navigation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 15 691–15 700.

[21] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, M. Deitke, K. Ehsani, D. Gordon, Y. Zhu *et al.*, "Ai2-thor: An interactive 3d environment for visual ai," *arXiv preprint arXiv:1712.05474*, 2017.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3736694

[22] H. Gao, Y. Yang, M. Sun, J. Wu, Y. Zhou, and B. Xu, "Clozemaster: Fuzzing rust compiler by harnessing llms for infilling masked real programs," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 712–712.

[23] R. Wang, M. Cheng, X. Xie, Y. Zhou, and L. Ma, "Moditector: Module-directed testing for autonomous driving systems," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 137–158, 2025.

[24] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, Z. Tian, Y. Huang, J. Hu, and Q. Wang, "Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model," in *Proceedings of the IEEE/ACM 46th International conference on software engineering*, 2024, pp. 1–12.

[25] X. Zhu, M. Jiang, X.-Y. Zhang, L. Nie, and Z. Ding, "Mtas: A reference-free approach for evaluating abstractive summarization systems," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2561–2583, 2024.

[26] Y. Xu, Y. Li, J. Wang, and X. Zhang, "Evaluating terminology translation in machine translation systems via metamorphic testing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 758–769.

[27] C. Xu, S. Chen, J. Wu, S.-C. Cheung, V. Terragni, H. Zhu, and J. Cao, "Mr-adopt: Automatic deduction of input transformation function for metamorphic testing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 557–569.

[28] X. Xie, X. Li, and S. Chen, "Metamorphic testing of image captioning systems via image-level reduction," *IEEE Transactions on Software Engineering*, 2024.

[29] M. Paltenghi and M. Pradel, "Morphq: Metamorphic testing of the qiskit quantum computing platform," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2413–2424.

[30] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[31] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 11 975–11 986.

[32] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, "Dinov2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024.

[33] C. Li, W. Tang, Y. Huang, S. S. Zhan, M. Hu, X. Jia, and Y. Liu, "Shedding light on vln robustness: A black-box framework for indoor lighting-based adversarial attack," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 1565–1574.

[34] L. Ma, F. Zhang, J. Sun, M. Xue, B. Li, F. Juefei-Xu, C. Xie, L. Li, Y. Liu, J. Zhao *et al.*, "Deepmutation: Mutation testing of deep learning systems," in *2018 IEEE 29th international symposium on software reliability engineering (ISSRE)*. IEEE, 2018, pp. 100–111.

[35] J. Ayerdi, P. Valle, S. Segura, A. Arrieta, G. Sagardui, and M. Arratibel, "Performance-driven metamorphic testing of cyber-physical systems," *IEEE Transactions on Reliability*, vol. 72, no. 2, pp. 827–845, 2022.

[36] J. Zhang, Z. Zheng, B. Yin, K. Qiu, and Y. Liu, "Testing graph searching based path planning algorithms by metamorphic testing," in *2019 IEEE 24th Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE, 2019, pp. 158–1580.

[37] P. Valle, S. Segura, S. Ali, and A. Arrieta, "Metamorphic testing of vision-language action-enabled robots," *arXiv preprint arXiv:2602.22579*, 2026.

[38] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *International Conference on Learning Representations (ICLR)*, 2019.

[39] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester, "Challenges of real-world reinforcement learning: definitions, benchmarks and analysis," *Machine Learning*, vol. 110, no. 9, pp. 2419–2468, 2021.

[40] W. Wu, H. He, J. He, Y. Wang, C. Duan, Z. Liu, Q. Li, and B. Zhou, "Metaurban: An embodied ai simulation platform for urban micromobility," in *International Conference on Learning Representations (ICLR)*, 2025.

[41] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[42] H. Sun, Z. Xu, M. Fang, Z. Peng, J. Guo, B. Dai, and B. Zhou, "Safe exploration by solving early terminated mdp," *arXiv preprint arXiv:2107.04200*, 2021.

[43] A. Ray, J. Achiam, and D. Amodei, "Benchmarking safe exploration in deep reinforcement learning," OpenAI, Tech. Rep., 2019.

[44] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of machine learning research*, vol. 22, no. 268, pp. 1–8, 2021.

[45] J. Ji, J. Zhou, B. Zhang, J. Dai, X. Pan, R. Sun, W. Huang, Y. Geng, M. Liu, and Y. Yang, "Omnisafe: An infrastructure for accelerating safe reinforcement learning research," *Journal of Machine Learning Research*, vol. 25, no. 285, pp. 1–6, 2024.

[46] A. Ng, M. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," *Advances in neural information processing systems*, vol. 14, 2001.

[47] K. R. Shahapure and C. Nicholas, "Cluster quality analysis using silhouette score," in *2020 IEEE 7th international conference on data science and advanced analytics (DSAA)*. IEEE, 2020, pp. 747–748.

[48] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[49] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

[50] S. Bensalem, K. Havelund, and A. Orlandini, "Verification and validation meet planning and scheduling," *International Journal on Software Tools for Technology Transfer*, vol. 16, no. 1, pp. 1–12, 2014.

[51] A. Orlandini, G. Bernardi, A. Cesta, and A. Finzi, "Planning meets verification and validation in a knowledge engineering environment," *Intelligenza Artificiale*, vol. 8, no. 1, pp. 87–100, 2014.

[52] Y. Li, J. S. Dong, J. Sun, Y. Liu, and J. Sun, "Model checking approach to automated planning," *Formal Methods in System Design*, vol. 44, no. 2, pp. 176–202, 2014.

[53] H. F. Eniser, T. P. Gros, V. Wüstholz, J. Hoffmann, and M. Christakis, "Metamorphic relations via relaxations: An approach to obtain oracles for action-policy testing," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 52–63.

[54] F. Percassi, E. Scala, and M. Vallati, "The power of reformulation: from validation to planning in pddl+," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 32, 2022, pp. 288–296.

[55] P. Smirnov, F. Joublin, A. Ceravola, and M. Gienger, "Generating consistent pddl domains with large language models," *arXiv preprint arXiv:2404.07751*, 2024.

[56] M. C. Magnaguagno, R. F. Pereira, M. D. Móre, and F. Meneguzzi, "Web planner: A tool to develop, visualize, and test classical planning domains," in *Knowledge Engineering Tools and Techniques for AI Planning*. Springer, 2020, pp. 209–227.

[57] G. P. Farias, R. F. Pereira, L. Hilgert, F. Meneguzzi, R. Vieira, and R. H. Bordini, "Predicting plan failure by monitoring action sequences and duration," *ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal*, vol. 6, no. 4, pp. 55–69, 2017.

[58] M. Cheng, Y. Zhou, X. Xie, J. Wang, G. Meng, and K. Yang, "Decictor: Towards evaluating the robustness of decision-making in autonomous driving systems," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[59] Q. Mazouni, A. Gotlieb, H. Spieker, M. Acher, and B. Combemale, "Mutation-guided metamorphic testing of optimality in ai planning," *Software Testing, Verification and Reliability*, vol. 35, no. 1, p. e1898, 2025.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on October 20,2026 at 14:23:28 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.